1. Problem Title & Link

Title: **LeetCode 410: Split Array Largest Sum**

Link: https://leetcode.com/problems/split-array-largest-sum/

2. Problem Statement (Short Summary)

You are given:

- An integer array nums
- An integer k

Your task is to **split the array into k non-empty contiguous subarrays**.

Among those k subarrays:

- Compute the **sum of each subarray**
- Take the **largest sum among them**

Goal:

Return the **minimum possible value of that largest subarray sum**.

In simple words:

Split the array into k parts such that

the largest subarray sum is as small as possible.

3. Examples (Input → Output)

Example 1

Input

nums = [7,2,5,10,8]

k = 2

Output

18

Explanation

Possible splits:

[7,2,5] [10,8] → max = 18

[7,2] [5,10,8] → max = 23

Minimum largest sum = **18**

Example 2

Input

nums = [1,2,3,4,5]

k = 2

Output

9

Explanation

Best split:

[1,2,3] [4,5]

Largest sum = **9**

Example 3

Input

nums = [1,4,4]

k = 3

Output

4

Explanation

Split:

[1] [4] [4]

Largest sum = **4**

4. Constraints

$1 \le$ nums.length ≤ 1000

$0 \le$ nums[i] $\le 10^6$

$1 \le k \le \min(50,$ nums.length$)$

Observations:

- Array must remain **contiguous**
- Need to minimize **maximum subarray sum**

5. Core Concept (Pattern / Topic)

Binary Search on Answer

This is the same pattern as:

LC 875 Koko Eating Bananas

LC 1011 Capacity to Ship Packages

LC 410 Split Array Largest Sum

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force Idea

Try all possible ways to split the array into k parts.

For each split:

- Calculate the largest subarray sum
- Track the minimum possible value

But number of splits grows exponentially.

Time complexity becomes extremely large.

Step 2 — Key Observation

The answer must lie between:

Lower bound

max(nums)

Because each subarray must contain at least one element.

Upper bound

sum(nums)

Because we could keep everything in one subarray.

Step 3 — Monotonic Property

If we try a maximum allowed sum:

Example

maxSum = 18

We check if we can split the array into **≤ k subarrays**.

If possible:

try smaller maxSum

If not possible:

increase maxSum

This monotonic property allows **binary search**.

7. Visual / Intuition Diagram (ASCII)

Example

nums = [7,2,5,10,8]

k = 2

Search range

10 → 32

Binary search

mid = 21

possible → yes

mid = 15

possible → no

mid = 18

possible → yes

Answer = **18**

8. Pseudocode (Language Independent)

```
low = max(nums)
high = sum(nums)

while low <= high:

    mid = (low + high) // 2

    if canSplit(mid):

        high = mid - 1

    else:

        low = mid + 1

return low
Helper function:
canSplit(maxSum):

    currentSum = 0
    subarrays = 1

    for num in nums:

        if currentSum + num > maxSum:

            subarrays += 1
            currentSum = 0

        currentSum += num

    return subarrays <= k
```

9. Code Implementation

Python

```
class Solution:
    def splitArray(self, nums, k):

        def canSplit(maxSum):

            current = 0
            parts = 1

            for num in nums:

                if current + num > maxSum:
                    parts += 1
                    current = 0

                current += num

            return parts <= k

        left = max(nums)
        right = sum(nums)

        while left <= right:

            mid = (left + right) // 2

            if canSplit(mid):
                right = mid - 1
            else:
                left = mid + 1

        return left
```

Java

```
class Solution {
```

```
public int splitArray(int[] nums, int k) {

    int left = 0, right = 0;

    for(int num : nums){
        left = Math.max(left, num);
        right += num;
    }

    while(left <= right){

        int mid = left + (right - left) / 2;

        if(canSplit(nums, mid, k)){
            right = mid - 1;
        }
        else{
            left = mid + 1;
        }
    }

    return left;
}

private boolean canSplit(int[] nums, int maxSum, int k){

    int current = 0;
    int parts = 1;

    for(int num : nums){

        if(current + num > maxSum){
            parts++;
            current = 0;
        }

        current += num;
```

```
        }

        return parts <= k;
    }
}
```

10. Time & Space Complexity

Time Complexity

Binary search iterations

O(log(sum(nums)))

Each iteration scans the array

O(n)

Total

O(n log(sum(nums)))

Space Complexity

O(1)

11. Common Mistakes / Edge Cases

1. Using wrong lower bound

should be max(nums)

2. Forgetting contiguous constraint
3. Incorrect subarray counting
4. Infinite binary search loop

12. Detailed Dry Run

Example

nums = [7,2,5,10,8]

k = 2

Range

low = 10

high = 32

Iteration 1

mid = 21

Split

[7,2,5] = 14

[10,8] = 18

Possible.

Iteration 2

mid = 15

Split

[7,2,5] = 14

[10] = 10

[8] = 8

3 parts → invalid.

Final answer

18

13. Common Use Cases (Real-Life / Interview)

Binary search on answer is used in:

- Workload balancing
- Server request distribution
- Partition optimization
- Manufacturing batch planning

14. Common Traps (Important!)

1. Confusing binary search index vs answer
2. Wrong split counting
3. Using greedy incorrectly
4. Incorrect mid update

15. Builds To (Related LeetCode Problems)

Binary Search on Answer progression

LC 875 Koko Eating Bananas

LC 1011 Capacity to Ship Packages

LC 410 Split Array Largest Sum

LC 1482 Minimum Days to Make Bouquets

LC 1283 Smallest Divisor Given Threshold

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Dynamic Programming	$O(n^2k)$	heavy

Binary Search + Greedy	O(n log sum)	optimal

17. Why This Solution Works

If a maximum subarray sum X works, then any value **greater than X will also work**.

This monotonic behavior allows binary search to efficiently find the **minimum valid maximum sum**.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Split into **exactly k subarrays**
2. Maximize the **minimum subarray sum**
3. Partition with **balanced loads**
4. Streaming partition problem